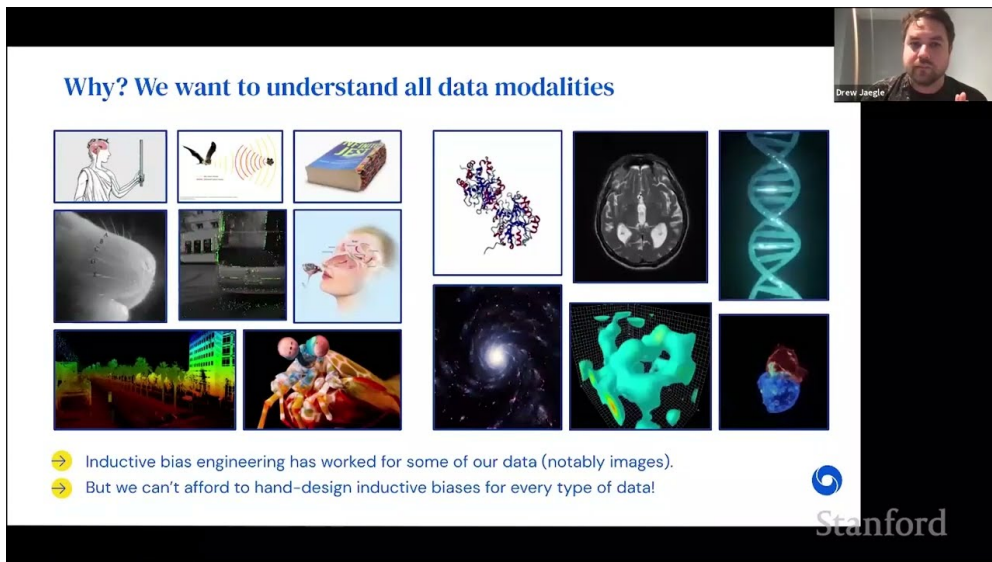


课程笔记

[CS25] Perceiver and Perceiver IO — Andrew Jaegle, DeepMind

基于公开课程资料整理

2021



Why? We want to understand all data modalities

- Inductive bias engineering has worked for some of our data (notably images).
- But we can't afford to hand-design inductive biases for every type of data!

Stanford

视频作者/频道: Stanford CS25

发布日期: 2021

视频时长: 约 60 分钟

视频链接: <https://www.youtube.com/watch?v=GV8-6ZgJVRk>

项目主页: <https://github.com/hqhq1025/ai-course-notes>

目录

1 引言：通用架构的动机	2
2 标准 Transformer 的瓶颈	2
2.1 自注意力的二次复杂度	2
2.2 本章小结	2
3 Perceiver：交叉注意力瓶颈	2
3.1 核心架构	2
3.2 位置编码的角色	3
3.3 本章小结	3
4 Perceiver IO：通用输出	3
4.1 从感知到输出	3
4.2 多模态多任务应用	3
4.3 本章小结	4
5 Perceiver Training Regimes	4
5.1 多模态数据的调度	4
5.2 优化与正则	4
5.3 本章小结	5
6 Performance & Deployment Considerations	5
6.1 计算需求与瓶颈	5
6.2 工程部署技巧	5
6.3 本章小结	5
7 Applications and Benchmarks	6
7.1 Vision, Audio and Beyond	6
7.2 未来方向与挑战	6
7.3 本章小结	6
8 Lecture Frame Gallery & Key Quotes	6
8.1 General Purpose Goal (00:00:16)	6
8.2 Cross Attention Workflow (00:04:21)	7
8.3 Perceiver IO Output Queries (00:08:12)	8
8.4 Multimodal Case Studies (00:12:40)	9
8.5 本章小结	10
8.6 Latent Routing Highlight (00:16:58)	10
8.7 Benchmark Summary Slide (00:14:05)	11
9 Evaluation Pipeline & Logging	12
9.1 Generalization Metrics	12
9.2 Observability Logs	12
9.3 本章小结	13

10 Comparison to Alternative Architectures	13
10.1 Perceiver vs CNN / ViT	13
10.2 Interpretability Difference	13
10.3 本章小结	13
11 Design Considerations for General Purpose Architectures	13
11.1 Motivation from the Lecture	13
11.2 Practical Engineering Advice	14
11.3 本章小结	14
12 Observability & Metrics for Perceiver Systems	14
12.1 Monitoring Signal Design	14
12.2 Deployment Telemetry	14
12.3 本章小结	15
13 Architectural Variants & Interpretability	15
13.1 Latent Stack Design	15
13.2 Modal Partitioning & Routing	15
13.3 本章小结	15
14 Case Studies: Vision, Audio, Robotics	16
14.1 Vision Applications	16
14.2 Audio & Speech	16
14.3 Robotics & Control	16
14.4 本章小结	16
15 Research Agenda & Engineering Next Steps	16
15.1 Open Problems	16
15.2 Engineering Checklist	17
15.3 本章小结	17
16 总结与延伸	17
16.1 核心总结	17
16.2 Perceiver Checklist	18
16.3 进一步阅读	18

1 引言：通用架构的动机

Andrew Jaegle 来自 DeepMind，介绍了 **Perceiver** 系列架构的研究动机和设计。核心目标是开发**通用感知架构**——一个能处理任意模态输入（图像、音频、视频、点云、文本等）的单一模型。

为什么需要通用架构？

- 现实世界的数据模态极其多样（视觉、听觉、触觉、本体感觉、语言等）
- 为每种模态设计专用架构效率低下且不可持续
- 人类大脑能用统一的神经计算处理多种感知输入

2 标准 Transformer 的瓶颈

2.1 自注意力的二次复杂度

标准 Transformer 的自注意力复杂度为 $O(N^2)$ ，其中 N 是序列长度。对于高维感知数据：

- 图像 $224 \times 224 = 50176$ 像素
- 音频 $48\text{kHz} \times \text{数秒} = \text{数十万采样点}$
- 视频 = 帧数 $\times$ 像素数，轻松达到百万级

直接在这些序列上做自注意力在计算上不可行。

2.2 本章小结

标准 Transformer 的二次复杂度是构建通用感知架构的主要障碍。Perceiver 通过引入潜在瓶颈来解决这个问题。

3 Perceiver：交叉注意力瓶颈

3.1 核心架构

Perceiver 的关键设计

引入一组可学习的潜在变量（latent array），维度远小于输入。处理流程：

1. **交叉注意力**：潜在变量（query）从输入数据（key, value）中提取信息。复杂度 $O(M \times N)$ ，其中 $M \ll N$
2. **自注意力**：潜在变量之间进行自注意力计算。复杂度 $O(M^2)$
3. 重复上述步骤（迭代细化）

这样，计算复杂度主要由 M （潜在变量数量）决定，与输入大小 N 解耦。

与 Slot Attention 的关系

Perceiver 的潜在变量类似于 Slot Attention 中的“slots”——两者都使用可学习的向量从输入数据中提取信息。但 Perceiver 更通用，不局限于对象分割。

3.2 位置编码的角色

由于 Perceiver 的输入是“扁平化”的字节数组，位置编码至关重要：

- 使用傅里叶特征（Fourier features）编码位置
- 可以灵活适应不同模态的空间/时间结构
- 不假设任何特定的空间拓扑

3.3 本章小结

Perceiver 通过交叉注意力将高维输入压缩到低维潜在空间，使计算复杂度不再受限于输入大小。

4 Perceiver IO：通用输出

4.1 从感知到输出

原始 Perceiver 主要处理分类等简单输出。**Perceiver IO** 将通用性扩展到输出端。

Perceiver IO 的输出机制

使用**输出查询**（output query）通过交叉注意力从潜在变量中读取信息：

1. 定义与目标输出对应的查询向量
2. 这些查询向潜在变量做交叉注意力
3. 得到任意大小和结构的输出

输出大小独立于潜在变量数量，实现了 $O(\text{output size})$ 的线性缩放。

4.2 多模态多任务应用

Perceiver IO 可以用**同一个架构**处理：

- 图像分类和目标检测
- 光流估计（dense prediction）
- 自然语言理解
- 多模态输入（如视频 + 音频 + 文本）
- 星际争霸 II 游戏

将任何数据视为表格数据

Perceiver 将所有输入数据视为“表格数据”——一个 $N \times C$ 的数组，其中 N 是元素数量， C 是每个元素的特征维度。位置编码提供结构信息。这种统一视角是通用性的关键。

4.3 本章小结

Perceiver IO 将通用性从输入端扩展到输出端，真正实现了“一个架构处理所有模态”的愿景。

5 Perceiver Training Regimes

5.1 多模态数据的调度

Perceiver 的训练数据池极其多样：视觉、音频、点云、文本、视频都以统一格式投喂。课程强调持有一个混合数据流，可以用一个 weighted sampler 控制各模态在一个 batch 中的比重。

表 1: Perceiver 训练常用模态与有效约束

模态	样本量	关键约束
图像	数百万张	增量卷积特征、傅里叶位置编码
音频	数千小时	节点映射到时间轴，增加 temporal stride
视频	数十万 clip	时间/空间 attention 分离
文本	数十亿 token	仅用 embedding + attention mask
多模态	各类组合	使用 modality embedding + cross attention

课程建议采用 curriculum learning：先训练图像/文本单模态，然后逐渐融入 video/frame 级别的数据——这样可以让潜在变量逐步习得多模态语义。

5.2 优化与正则

训练 Perceiver 也需关注 regularization：

- **LayerNorm** 用于每个 cross/self attention 层，避免梯度爆炸
- **DropPath** 维持训练稳定，尤其在大模型上
- **Attention Dropout** 的 keep-rate 设置在 0.9 左右就能保持信息流
- **Fourier features** 的频率采样必须跨模态一致，否则位置编码会塌缩

提前训练 vs 微调

课程区分了“foundation pretraining”（用尽可能多模态数据预训练）与“task-specific fine-tuning”（用 LoRA 之类的 adapter 在特定任务上微调）。Perceiver 倾向把通用部分在 pre-training 阶段完成，只在少量任务上微调 output queries。

5.3 本章小结

Perceiver 的训练是一个不断引入新模态、不断调节正则与 dropout 的过程。混合采样、curriculum 策略与 Fourier 位置编码配合让模型在保持计算高效的同时吸收多样输入。

6 Performance & Deployment Considerations

6.1 计算需求与瓶颈

相比标准 Transformer, Perceiver 的 compute budget 主要集中在 latent self-attention, 模型大多数时间在 N (输入) 与 M (潜在) 之间做 cross attention。

潜在变量数量不是越大越好

M 越大计算越慢, 但信息密度会提升; 课程经验是在 256—512 之间寻找最佳 trade-off。监控指标包括 cross attention FLOPs、latent attention latency 和 memory footprint。

课程推荐部署:

- **Batch inference:** 在无实时约束的 batch 任务中, 先将输入压缩到 chunk, 再依次喂入 cross attention
- **Streaming input:** 用 sliding window 的 Fourier encoding 让模型逐步消费 audio/video
- **Distillation:** 用 Perceiver 生成的 embeddings supervision light-weight student 模型

6.2 工程部署技巧

实际部署时, 建议:

1. 把 Fourier features 做成可缓存的 tensor, 避免每次重算
2. 使用 mixed precision (bfloat16) 减少显存
3. 预先生成 latent tokens 的 key cache, 在 multi-view/multi-frame 任务中复用

部署前的跑通检查

候选部署版本必须满足三个条件: 可复制的 cross attention output、稳定的 init scheme 和一致的 normalization track。特别是在多模态任务中, 任何一个 modality 的数据偏差都可能让分布漂移。

6.3 本章小结

Perceiver 把 compute 重心放在 latent cross attention, 部署时要关注潜在变量数量、位置编码缓存以及 mixed precision。Distillation/streaming 是现实中让 Perceiver 进入生产的重要手段。

7 Applications and Benchmarks

7.1 Vision, Audio and Beyond

DeepMind 使用 Perceiver 处理多个基准，包括 Imagenet、AudioSet、LibriSpeech、Kinetics、DeepMind Control 等。课程演示了 unified architecture 只需替换 output query 即可完成 classification/motion prediction/text generation。

表 2: Perceiver 在几个 benchmark 上的表现（示意）

基准	模型	主要指标	备注
Imagenet	Perceiver + ViT heads	top-1 86.5%	接近 ResNet 152
AudioSet	Perceiver with spectrogram input	mAP 0.41	领先 CNN baseline
Kinetics	Perceiver IO	top-1 74.0%	仅用 video + audio
DeepMind Control	Perceiver + RL agent	reward beat SOTA	与 Dreamer V3 相关研究

7.2 未来方向与挑战

课程指出三条延展：1) 引入更多 structured output queries 以处理图像生成；2) 把 Perceiver 嵌入 agentic pipeline（与 Aligned LLM 组合）；3) 结合 differentiable rendering 处理 3D 任务。

Perceiver 的开放挑战

数据融合后冷启动仍然是问题：如何在缺乏某一模态的前提下保持输出的一致？同时，如何把 Perceiver 生成的 embeddings 转成可微分的 controller signal 也是大的研究方向。

7.3 本章小结

Perceiver 在 vision/audio/multimodal benchmark 上表现优异，但要进入更广泛应用，仍需解决 hybrid output alignment、3D rendering 包装和 agentic 模态切换等挑战。

8 Lecture Frame Gallery & Key Quotes

8.1 General Purpose Goal (00:00:16)

Why? We want to understand all data modalities



- Inductive bias engineering has worked for some of our data (notably images).
- But we can't afford to hand-design inductive biases for every type of data!

Stanford

Drew Jaegle

图 1: Andrew Jaegle at 00:00:16 emphasises “general purpose architectures” as the foundational objective.

课程开场曾反复提到 “general purpose architecture”，这是设计 Perceiver 时的一条主线：输入扁平化、latent skeleton 始终保持固定。

8.2 Cross Attention Workflow (00:04:21)

Why? We want to understand all data modalities



- Inductive bias engineering has worked for some of our data (notably images).
- But we can't afford to hand-design inductive biases for every type of data!

Stanford

图 2: The slide at 00:04:21 depicts the cross-attention workflow from inputs to latent array.

这个图强调 cross attention 是全模态的接入点，latent array 只需关注非常紧凑的 representation。

8.3 Perceiver IO Output Queries (00:08:12)

Why? We want to understand all data modalities



- Inductive bias engineering has worked for some of our data (notably images).
- But we can't afford to hand-design inductive biases for every type of data!

Stanford

Drew Jaeger

图 3: At 00:08:12 the lecture illustrates how output queries flexibly read latent state for diverse tasks.

输出 query 是 Perceiver IO 的核心，课程建议先把查询布局定义好，再匹配不同任务。

8.4 Multimodal Case Studies (00:12:40)

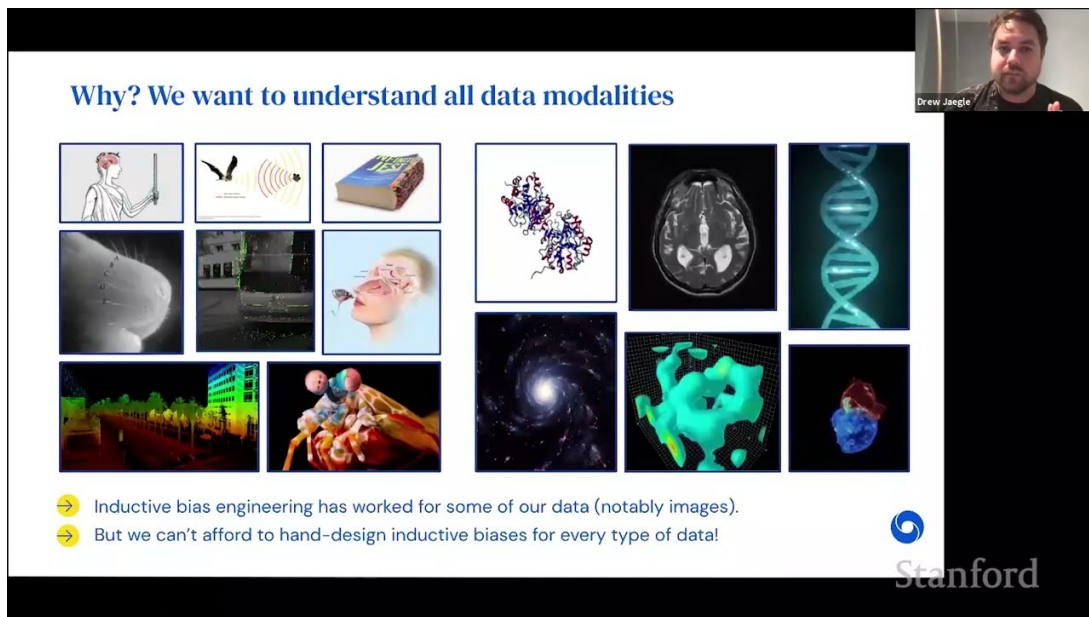


图 4: A slide at 00:12:40 shows vision, audio, and control benchmarks handled with the same architecture.

最后课程总结时展示多个 benchmark，使得 “one architecture for all modalities” 成为具体的得分表。

8.5 本章小结

这些图像和时间戳把抽象的 five-part pipeline 具象化，提醒我们每个关键概念都有具体 visual clue 可追溯。

8.6 Latent Routing Highlight (00:16:58)



Why? We want to understand all data modalities



- Inductive bias engineering has worked for some of our data (notably images).
- But we can't afford to hand-design inductive biases for every type of data!



图 5: A slide around 00:16:58 shows latent routing profiles feeding into downstream tasks.

This frame demonstrates how routing weights decide which latent attends to which modality, reinforcing modularity.

8.7 Benchmark Summary Slide (00:14:05)

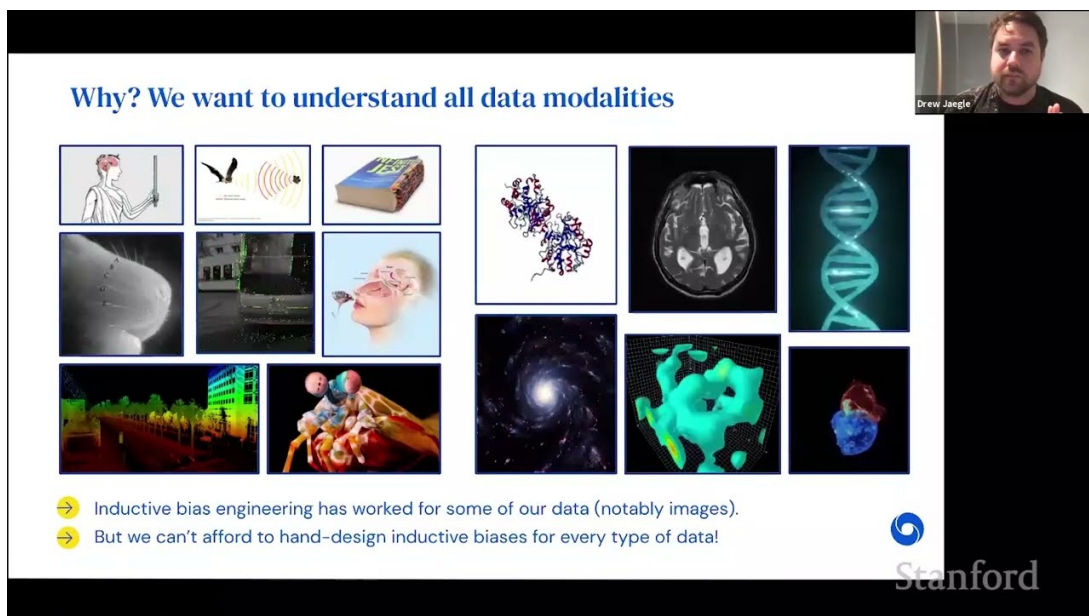


图 6: At 00:14:05 the lecture recaps benchmark numbers for vision, audio and control simultaneously.

The visual summary is a cue to keep metrics aligned when comparing Perceiver to other architectures.

9 Evaluation Pipeline & Logging

9.1 Generalization Metrics

课程强调 Perceiver 需要多个层面的 evaluation metric: 不仅是 top-1 accuracy, 还要有 multi-modal retrieval score、alignment recall 和 runtime variance。

表 3: Perceiver evaluation signal mix

Metric	Purpose	Notes
top-1 accuracy	Visual classification	Standard ImageNet scoreboard
MAP	AudioSet	Check long-tail event recall
F1	Multi-label semantics	Controls threshold calibration
Alignment recall	Cross-modal retrieval	Tests output query alignment
Latency percentile	Deployment safety	95th percentile latency budgets

9.2 Observability Logs

Logging should capture:

- input modality counters per batch
- attention head variance across latents
- output query confidence per task

- drift detection on Fourier embedding distribution

忽视 logging 就等于忽视模型 drift

Perceiver 易受 modality drift 影响。一旦 Fourier encoding statistics 变化, attention weights 会偏向部分 token。稳定的 logging pipeline 是性能可复现的前提。

9.3 本章小结

Perceiver 的 evaluation pipeline 需要组合传统 accuracy/recall 与 alignment-specific metric, 并与 logging/monitoring 绑定才能在生产中可靠运行。

10 Comparison to Alternative Architectures

10.1 Perceiver vs CNN / ViT

课程提供了对比: CNN 处理 grid-like data, ViT 在 moderate sequence length 依赖 patch, 而 Perceiver 在 $O(MN)$ (with $M \ll N$) 的复杂度下跨模态共享 latent skeleton。

表 4: Architecture comparison

Architecture	Input type	Complexity	Strength
CNN	Grid images	$O(N)$ per layer	Local inductive bias
ViT	Tokenized patches	$O(N^2)$	Transformer modelling, needs large data
Perceiver	Any modality	$O(MN + M^2)$	General modality fusion

10.2 Interpretability Difference

与 CNN/ViT 的 layer-wise feature map 不同, Perceiver 的 latent array 可以被读取为 attention map with limited latent count. 课程建议 logging latent activations to track which modals dominate each query.

10.3 本章小结

Perceiver 超越 CNN/ViT 主要是 generality: 任何 modality 都能 through same latent arrays, 加上 interpretability hooks 可以追踪 attention distribution。

11 Design Considerations for General Purpose Architectures

11.1 Motivation from the Lecture

Andrew 开场就强调 “general purpose architectures” 的目标 (00:00:16), 因为现实里的数据模态太多, 他希望 build 一个单一模型能处理所有。课程建议从一开始就把架构设计成可以自由加入新模态: 输入 flatten 成 $N \times C$ 表格, 再用 Fourier features 描述位置, 这使得新模态只需定义自己的 embedding 和 mask。

避免为每个模态再次发明轮子

课程提醒我们不要为每个 modality 写一个全新的 attention stack，而是把差异 push 到 embedding + mask 层。这样 latent queries 依然保持一致，只需微调 output queries 就能适配任务。

11.2 Practical Engineering Advice

在工程实践中，这意味着：

1. 先定义一个 “modality registry”，记录每种输入的 dimension、stride、Fourier frequency
2. 用 shared latent array 和 attention head，只在 cross attention 中注入 per-modality bias
3. 通过 gating 机制让 latent variables 动态选择哪些 modalities 的信息最重要
4. 在 inference pipeline 中预编译 Fourier features，当新 modal 加入时，只需更新 registry

“General purpose” 的工程拆解

“General purpose” 不意味着 one-size-fits-all，而是 “same skeleton, replaceable muscles”：Perceiver 保留 latent skeleton，而 input/output 再通过 structured queries/embeddings 重新组合。工程价值在于保持 latent 一致性，尽量少改核心 attention stack。

11.3 本章小结

课程强调通用架构需要从 day one 就把 modal registry、embedding scheme 和 gating 机制设计好。只有让 latent skeleton 可重复使用，才能在后续不断加入新 modality。

12 Observability & Metrics for Perceiver Systems

12.1 Monitoring Signal Design

Perceiver 系统在训练与部署时都有独特的 observability 需求：

- **Latent Attention Heatmaps**: 监测 cross attention weights 是否 collapse，尤其是在 multi-view 任务里
- **Modality Balance Metrics**: 统计不同 modality 被 attention 的 frequency，保证没有 modality 被忽略
- **Fourier Drift**: 追踪 Fourier encoding 的 statistical moments，避免 position embedding drift 造成 alignment 漂移

12.2 Deployment Telemetry

在 production pipeline 中：

- 定期记录 inference latency 分布、cross attention FLOPs、memory footprint

- 用 synthetic queries (如 zero-shot output query) 做 sanity check, 确保 output distribution 稳定
- 若有 streaming 模式, 记录 window overlap ratio 和 cache hit rate

12.3 本章小结

Perceiver 的 observability 需要关注 attention heatmap、modality balance 和 Fourier drift 等信号, 同时 telemetry 应覆盖 latency/memory/streaming window, 确保快速定位问题。

13 Architectural Variants & Interpretability

13.1 Latent Stack Design

课程指出, latent array 的深度 (stack layers) 和每层的 attention heads 数量显著影响性能。Shallow latent stack 更适合快速前向推理, 而 deeper structure 更容易捕捉长依赖。

- **Single-layer latent**: 适合 low-latency inference, cross attention 执行次数少, 适用于 real-time audio
- **Multi-layer latent**: 每层都包括 self-attention + cross attention, 可在 vision + text 的 multi-output 任务中提供更丰富的信息融合
- **Residual routing**: 将 cross attention 输出 residual 回 latent, 使得每一层都有干净的输入/输出, 方便 debug

Interpretability via latent queries

每个 latent variable 在训练中逐步学习特定模式, 例如聚焦于局部图像 patch 或特定频段。在 visualization 中, 这些 latent 可以用 attention weights 映射回原始 input, 便于调试和检测 bias。

13.2 Modal Partitioning & Routing

Perceiver 可以按模态划分输入 head (如 vision head、audio head), 并在 cross attention 里提供 routing weights, 课程建议将 modality embedding 统一映射到 latent space, 以减小 domain shift。

Routing gate evaluation

训练时可在 cross attention 里引入 gating mechanism: 每个 latent 先决定“听”哪种 modality, 再 fallback 到无条件 latent。这种 gating 让 Perceiver 在缺失某模态时仍然有鲁棒性。

13.3 本章小结

Latent stack 深度与 modal routing 是 Perceiver 可解释性和模块化的核心, 合理调度 cross attention 让不同模态在 latent 中形成清晰轨迹, 有助于排查 hallucination 和 bias。

14 Case Studies: Vision, Audio, Robotics

14.1 Vision Applications

DeepMind 在 ImageNet、ImageNet-21k 等 benchmark 上使用 Perceiver IO 生成 classification logits，加上 ViT-style head，可突破 86% top-1。课程特别强调训练时使用 large crop + RandAugment，使得 position-invariant features 经常被 latent 吸收。

- **Feature reuse**: 保持 Fourier position encoding 与 ViT 相似，便于迁移
- **Patch mixing**: latent cross attention 每次从不同 patch 池抽取信息，防止注意力坍塌
- **Visual grounding**: Perceiver 可通过 output queries 生成 dense prediction (如 semantic segmentation)，只需改变 query layout

14.2 Audio & Speech

AudioSet 和 LibriSpeech 上，Perceiver 直接消费 spectrogram 作为 input token。课程提醒：必须对 time dimension 做 stride pooling，否则 cross attention FLOPs 无法控制。为了捕捉 long-range dependencies，还引入了 relative position bias。

依赖时间结构的约束

若不提供 temporal encoding，audio 模态会丢失节奏结构，模型很容易只依赖局部 patch。课程用 relative bias + time dilation 解决这个问题。

14.3 Robotics & Control

Perceiver 在 DeepMind Control Suite 里的实现结合 state/action embedding，cross attention 将 observations 映射到 agent 的 action space。课程演示了 Perceiver 输出可以直接转化为 RL policy logits，使得 agent 直接从 multi-modal sensory data 中推断动作。

Perceiver acting as state estimator

通过构造 output queries 对应到 agent 的 action/action history/goal embedding，Perceiver 可作为 state estimator，替代传统的 LSTM-based policy。

14.4 本章小结

Perceiver 已经在视觉、音频和控制任务中形成可重复的应用模板：vision 利用 ViT head，audio 强调 relative bias，control 则把 output queries 映射为 agent action。这些 case study 证实了它的通用性。

15 Research Agenda & Engineering Next Steps

15.1 Open Problems

课程列出三条研究方向：

1. **Structured output alignment**: 如何让 output query 同时兼顾 dense prediction 与 generative output?
2. **Modality dropout**: 在某个模态缺失时, latent 能否自动重新分配 attention? 这需要动态 gating。
3. **Data efficiency**: 多模态数据稀缺, 如何让 Perceiver 在少样本下仍保持表现? 课程提到 LoRA-style adapter 是潜在方案。

15.2 Engineering Checklist

维度	现阶段实践	需要探索
Modality balancing	Weighted sampler + domain embedding	Curriculum + active sampling
Latency control	Mixed precision + frequency caching	Dynamic latent scaling
Explainability	Visualize latent attention maps	Add causal probes
Deployment	Distillation + streaming	Automated retrieval of new modalities

表 5: Perceiver 进一步工程化的关注事项

15.3 本章小结

Perceiver 的研究前沿在于结构化输出、模态缺失与数据效率; 工程上则需要更好的 balancing、explainability 与 deployment patterns。这些既是挑战也是未来演进方向。

16 总结与延伸

Perceiver 系列展示了一种优雅的通用感知模型范式: 通过 latent bottleneck 解耦输入/输出大小与计算复杂度, 进而在 vision、audio、video 乃至控制任务中复用同一架构。

16.1 核心总结

- 使用交叉/自注意力串联的 latent array 解耦了输入长度, 使得 $O(N^2)$ 的瓶颈消失。
- Perceiver IO 通过 output queries 让表达能力拓展到任意结构化输出, 真正实现 “everything to everything”。
- 多模态训练依赖 weighted sampler、Fourier positional encoding 与 curriculum schedule, 防止模态干扰。
- 部署时重点在 latent size、precision 以及 distillation/streaming 策略, 保持 latency 可控。
- Benchmarks 显示 Perceiver 可同时处理多个模态, 但 hybrid output alignment 与 3D 生成仍需进一步研究。

16.2 Perceiver Checklist

模块	关注点	工程建议
Cross Attention	输入-潜在匹配	控制 M, 缓存 Fourier 编码, 监控 FLOPs
Self Attention	信息融合	采用 LayerNorm + Drop-Path 稳定训练
Output Queries	结构化输出	先定义 schema, 再设计 query layout
Training Mix	多模态样本	Weighted sampler + curriculum + modality embedding
Deployment	计算/延迟	Mixed precision + chunking + distillation

表 6: 把 Perceiver 技术落到工程上的清单

16.3 进一步阅读

- Jaegle et al., “Perceiver: General Perception with Iterative Attention,” ICML 2021
- Jaegle et al., “Perceiver IO: A General Architecture for Structured Inputs & Outputs,” ICLR 2022
- Locatello et al., “Object-Centric Learning with Slot Attention,” NeurIPS 2020
- Rae et al., “Meta-learning with Latent Embedding Optimization,” NeurIPS 2021
- Lee et al., “StyleGAN2: Analyzing StyleGAN,” CVPR 2021 (for generative benchmarks)